



What is RAG?

Retrieval-Augmented Generation — Give your LLM access to **your data**

✗ Without RAG

LLM only knows its training data. Ask about your company docs? It **hallucinates**.

✓ With RAG

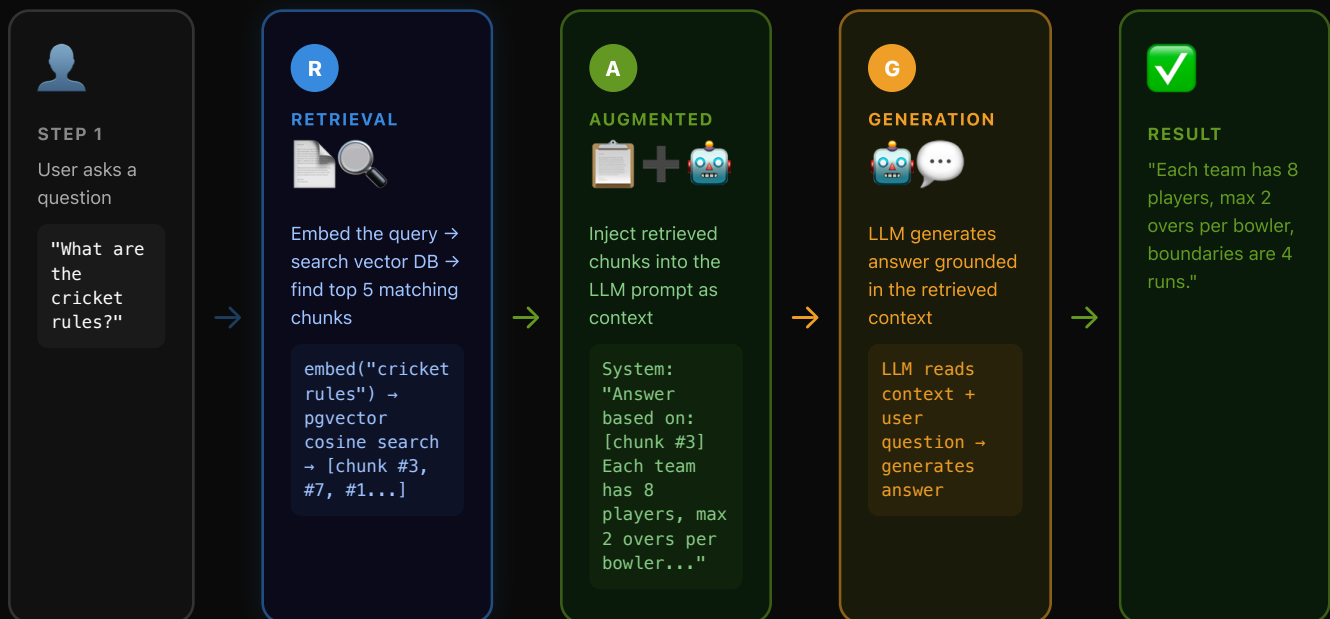
LLM gets **relevant context** from your documents before answering. Grounded, accurate.

User Question → **Retrieve Relevant Docs** → Inject as Context → **LLM Generates Answer**

💡 RAG = "Look it up, then answer" — like an open-book exam for AI

Retrieval · Augmented · Generation

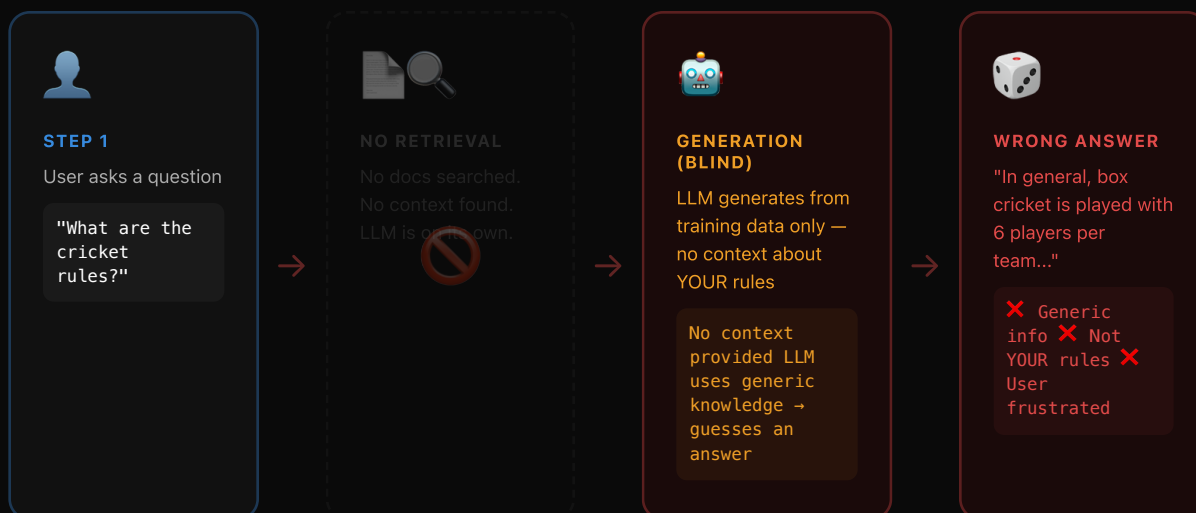
Example: "What are the box cricket rules for our tournament?"



💡 Retrieval finds the right docs · Augmented injects them as context · Generation produces a grounded answer

Without RAG — Just Generation

Same question: "What are the box cricket rules for our tournament?"



⚠️ Without Retrieval & Augmentation, the LLM has no access to your documents — it can only guess from generic training data.

What is a **Vector Database**?

A database that stores and searches data by **meaning**, not exact keywords



What it stores

High-dimensional vectors (arrays of numbers) that represent the **semantic meaning** of text, images, or any data.



How it searches

Finds the **most similar** vectors using distance metrics (cosine, euclidean) — not exact string matching.



Why it matters

"cricket rules" matches "batting regulations" because they're **semantically close** — impossible with SQL LIKE.

Traditional DB: `SELECT * WHERE text LIKE '%cricket%'` → exact match only

Vector DB: `Find nearest neighbors to embed("cricket rules")` → meaning-based match

Feature	Traditional DB	Vector DB
Search type	Keyword / exact match	Semantic similarity
Understands synonyms	No	Yes
Data format	Rows & columns	High-dimensional vectors
Example	PostgreSQL, MySQL	pgvector , Pinecone, Weaviate



We use **pgvector** — a PostgreSQL extension. No new infrastructure needed, just add vector columns to your existing DB.

How Embeddings Work

Text → Numbers that capture meaning

```
"King"    → [0.21, 0.85, 0.12, 0.67, ..., 0.43]    // 768 dimensions
"Queen"   → [0.19, 0.82, 0.14, 0.71, ..., 0.41]    // very similar!
"Apple"   → [0.91, 0.03, 0.78, 0.11, ..., 0.88]    // completely different
```



What's a dimension?

Each number captures one aspect of meaning: royalty, gender, power, food, etc.



How to compare?

Cosine similarity — measures the angle between two vectors. Closer = more similar.

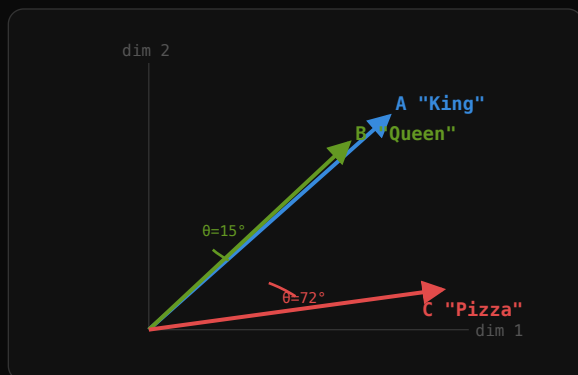


Who creates them?

Embedding models: **nomic-embed-text** (Ollama), OpenAI text-embedding-3-small

How are vectors compared? Cosine Similarity

Measures the [angle](#) between two vectors — not the distance



Smaller angle = more similar meaning

The Formula

$$\cos(\theta) = (A \cdot B) / (|A| \times |B|)$$

Result range: -1 to 1

1.0 = identical meaning

0.0 = unrelated

-1.0 = opposite meaning

Example Calculation

A = "King" → [0.9, 0.8]

B = "Queen" → [0.85, 0.9]

$$A \cdot B = (0.9 \times 0.85) + (0.8 \times 0.9) = 1.485$$

$$|A| = \sqrt{(0.81 + 0.64)} = 1.204$$

$$|B| = \sqrt{(0.72 + 0.81)} = 1.237$$

$$\cos(\theta) = 1.485 / (1.204 \times 1.237) = 0.997 \checkmark \text{ Very similar!}$$

Comparison	Angle	Cosine Similarity	Meaning
King ↔ Queen	~15°	0.97	Very similar
King ↔ Prince	~30°	0.87	Related
King ↔ Pizza	~72°	0.31	Unrelated

The "King" Example — Words in Vector Space

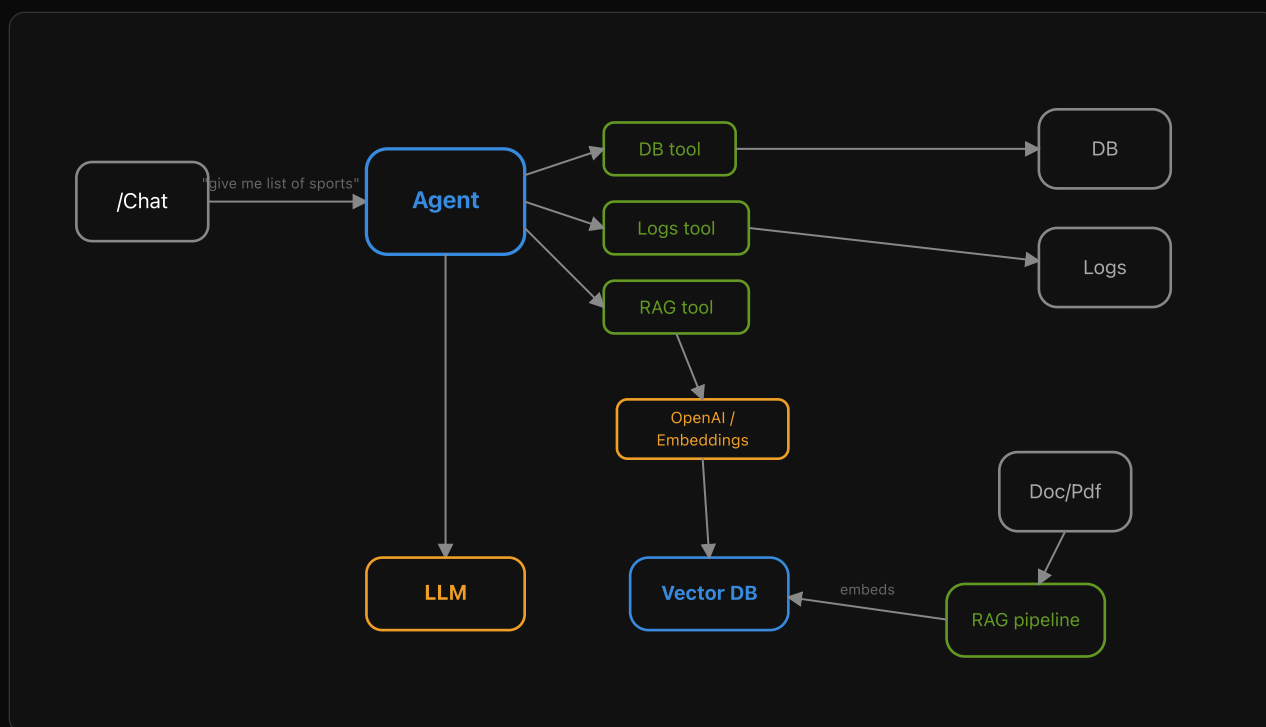
Words with similar meaning are **close together** in vector space



Word	Distance from "King"	Similarity
Queen	0.12	Very Close ●●●●○
Monarch	0.15	Close ●●●○○
Prince	0.28	Related ●●○○○
Apple	0.89	Unrelated ●○○○○

Phase 1: Single Agent Router

Single Agent picks a tool, executes, summarizes with LLM



💡 Simple but limited — LLM must choose from ALL tools at once. Gets confused with many options.

Phase 1: How It Works — "Give me list of sports"

What We're Achieving

- One **Agent** receives all user queries via `/chat`
- Agent asks the **LLM** to pick the right tool
- Tool executes, returns raw data
- LLM summarizes the result into a human-friendly answer

💡 The Agent is the brain — it decides **which tool** to call and **how** to present the result.

Example Flow: "Give me list of sports"

Step	What Happens
1	User sends <code>/chat</code> → "Give me list of sports"
2	Agent asks LLM: "Which tool?" → DB tool
3	DB tool runs query → <code>SELECT * FROM sports</code>
4	DB returns: Cricket, Football, Badminton, Chess
5	Agent sends data to LLM → "Summarize this"
6	LLM responds: "There are 4 sports: Cricket, Football, Badminton, Chess"

```
// Response
{ "response": "There are 4 sports available: Cricket, Football, Badminton, and Chess.",
  "toolUsed": "db_tool", "query": "SELECT name FROM sports" }
```

Phase 1: RAG Tool Flow — "What are the box cricket rules?"

When the question is about **documents**, the Agent picks the RAG tool

Step	Who	Action
1	User	Sends <code>/chat</code> → "What are the box cricket rules in our tournament?"
2	Agent	Asks LLM: "Which tool?" → RAG tool (question is about docs, not DB)
3	RAG tool	Embeds the query via OpenAI/Embeddings → gets vector [0.23, 0.87, ...]
4	Vector DB	Cosine similarity search → finds top 3 matching chunks from rules.pdf
5	RAG tool	Returns chunks: "Each team has 8 players, 6 overs, boundaries = 4 runs..."
6	Agent	Sends chunks + question to LLM → "Answer based on this context"
7	LLM	Generates grounded answer from the retrieved document chunks

```
// Response
{ "response": "According to the tournament rules: Each team has 8 players, the match is 6 overs per side, boundaries count as 4 runs, and max 2 overs per bowler.",
  "toolUsed": "rag_tool", "chunksUsed": 3, "source": "box-cricket-rules.pdf" }
```

💡 The Agent doesn't know the rules — it retrieves them from your uploaded PDF via vector search, then the LLM answers using that context.

🤔 **How does the LLM pick RAG tool over DB tool?** Each tool has a description:

- **db_tool**: "Query structured data — users, teams, sports"
- **rag_tool**: "Answer questions about documents, rules, policies, PDFs"

The LLM matches the user's intent to the best tool description semantically — not by keywords.

Phase 1: RAG Flow — End to End

Ingestion (one-time)

```
POST /api/ingest (file: rules.pdf)
↓
Extract text (PDFBox)
↓
Split into chunks (500 chars)
↓
Embed each chunk (Ollama)
↓
Store in pgvector
```

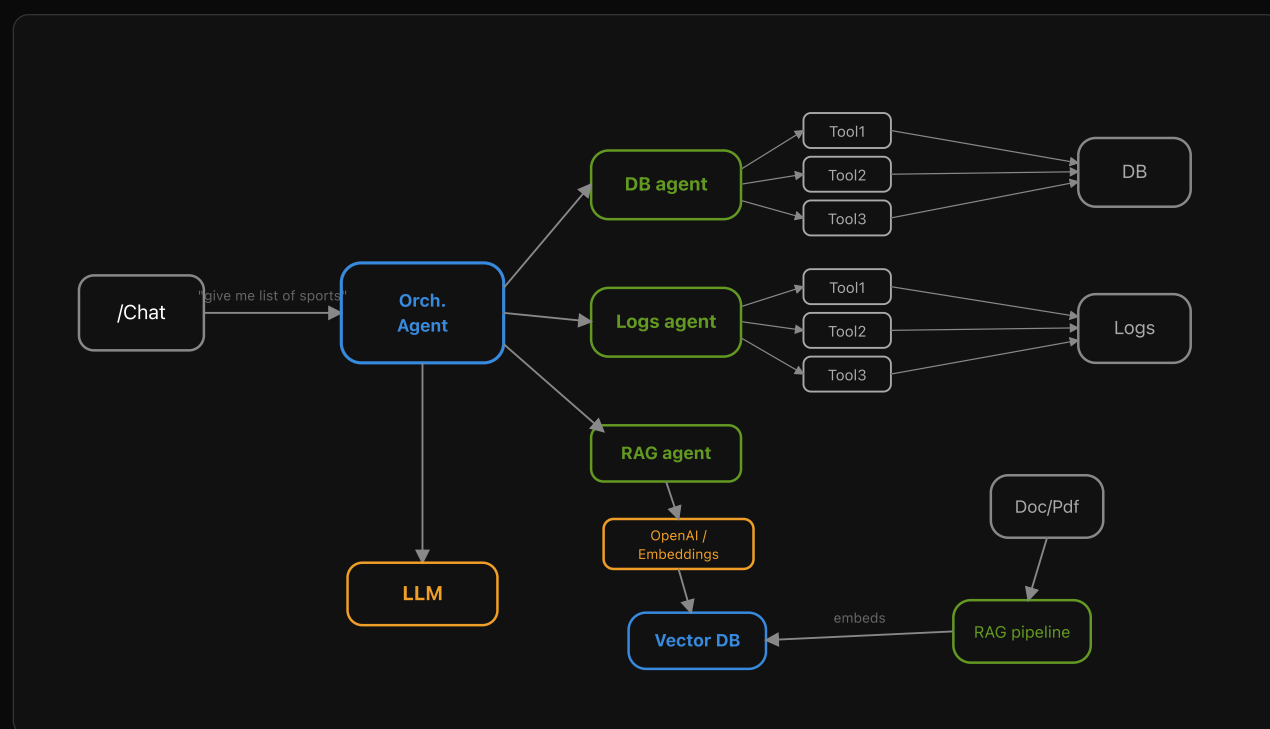
Query (every request)

```
POST /api/chat "What are the rules?"
↓
Embed question (Ollama)
↓
Search pgvector (top 5 similar)
↓
Pass chunks as context to LLM
↓
LLM answers from YOUR docs
```

💡 The LLM never sees the full document — only the most relevant chunks. This is how RAG scales to thousands of pages.

Phase 2: Multi-Agent Orchestrator

Each domain gets its own agent with specialized tools



Phase 2: What Changed?

From one overloaded agent to **specialized agents** with focused responsibilities

Orchestrator Agent

Only decides **which agent** to route to. One simple decision — not which tool from 9 options.

DB Agent

Owens all database tools (query users, teams, sports). Picks from **its own 3 tools** only.

Logs Agent

Owens log tools (search, filter by level, tail). Focused on **log operations** only.

RAG Agent

Handles document questions. Embeds → searches Vector DB → returns grounded answers.

Aspect	Phase 1 (Single Agent)	Phase 2 (Multi-Agent)
Decision complexity	1 agent picks from 9 tools	Each agent picks from 3 tools
Accuracy	LLM confused with many options	Simpler decisions = fewer mistakes
Adding new features	Add to god-service	Add a new agent
LLM calls per request	2	3 (route + pick tool + summarize)

💡 Trade-off: slightly more LLM calls, but each call is simpler and more accurate. Easier to extend and debug.

Phase 2: Examples

"Give me list of sports"

Step	Action
1	Orchestrator → LLM: "Which agent?" → DB agent
2	DB agent → LLM: "Which tool?" → query_sports
3	Tool runs: SELECT * FROM sports
4	DB agent → LLM: "Summarize" → response

```
{ "agentUsed": "db",  
  "toolUsed": "query_sports",  
  "response": "4 sports: Cricket, Football, Badminton, Chess" }
```

"What are the box cricket rules?"

Step	Action
1	Orchestrator → LLM: "Which agent?" → RAG agent
2	RAG agent embeds query → searches Vector DB
3	Top 3 chunks retrieved from rules.pdf
4	RAG agent → LLM + chunks → grounded answer

```
{ "agentUsed": "rag",  
  "chunksUsed": 3,  
  "response": "8 players per team, 6 overs, 4 runs per ball" }
```

💡 The Orchestrator only picks the agent (1 decision from 3 options). Each agent then handles its own domain independently.

Tech Stack



LLM

Ollama + llama3.1

Local, free, supports tool-calling



Embeddings

nomic-embed-text

768 dimensions, runs locally via Ollama



Vector DB

pgvector on PostgreSQL

No new infra — extension on existing DB



PDF Parsing

Apache PDFBox

Extract text, chunk, embed, store

Component	Phase 1	Phase 2
Architecture	Flat (1 service, 3 tools)	Multi-agent (orchestrator + 3 agents)
LLM calls/request	1-2	2-3
Accuracy	Good	Better (simpler decisions)
Extensibility	Hard (god service)	Easy (add agent/tool)

Key Takeaways

🧠 **RAG** = Retrieve relevant docs + feed to LLM as context. Open-book exam for AI.

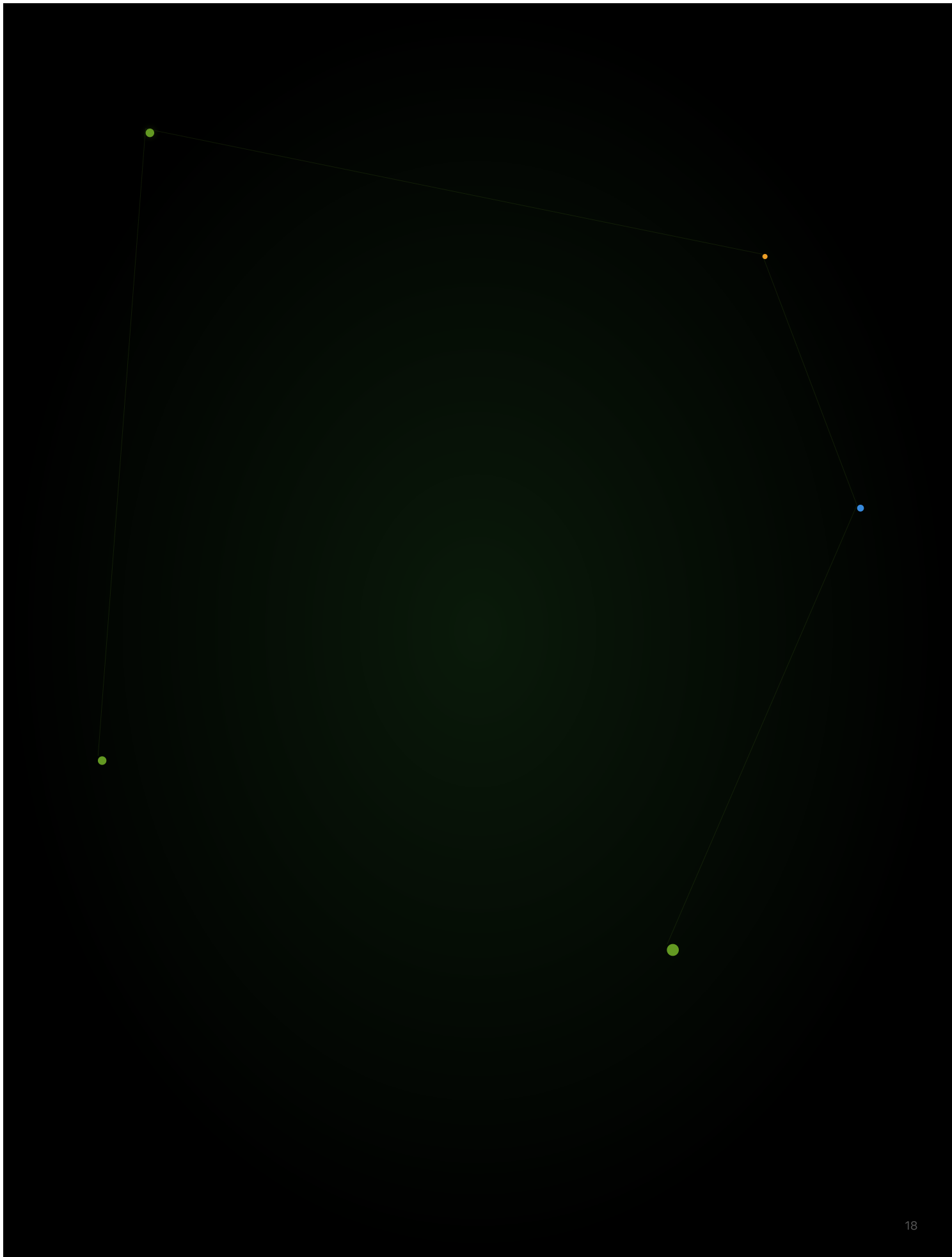
📐 **Vector DB** = Store meaning as numbers. Search by similarity, not keywords.

🔥 **Embeddings** = "King" is close to "Queen" but far from "Pizza" in vector space.

🛠️ **Agent Orchestrator** = Route to specialized agents. Each agent makes simpler, better decisions.

🆓 **All local** = Ollama + pgvector. No API keys, no cost, no rate limits.

Questions?



RAG has Multiple Retrieval Strategies

The "Retrieve" step in RAG is not one-size-fits-all. Vector search is just one option.



Dense Retrieval

(Vector)

Embed query + docs into vectors. Find nearest neighbours by cosine similarity.

Great for: semantic meaning, paraphrases, multilingual



Sparse Retrieval

(BM25)

Score docs by keyword frequency and rarity. No embeddings, no vectors.

Great for: exact terms, product names, IDs, code



Hybrid

Combine BM25 keyword scores with vector similarity scores for the best of both worlds.

Great for: large corpora needing both precision and recall



Sahil covered Dense (Vector) retrieval. Now let's look at where it struggles — and what to use instead.

Where Vector Search Fails

Embeddings are powerful — but they have real blind spots.

✗ Exact terms get lost

User searches "**error code E-4021**". The embedding model generalises it — returns docs about "error handling" instead of the specific code.

```
Query: "E-4021 payment failure"
Vector result: "handling payment issues"
BM25 result: "E-4021: card payment error"
```

✗ Expensive to set up

Every doc must be embedded and stored. Adding new docs means re-embedding. Needs a running embedding model 24/7.

```
// For 10,000 docs:
10,000 × embed() calls at inference
+ pgvector index to maintain
+ embedding model always running
```

⚠ Overkill for small corpora

200 FAQs? 50 wiki pages? Full vector infrastructure is more complexity than the problem warrants.

```
// Reality check
200 FAQs × 768 dims = 1.2M floats
BM25 on same data = a few KB
Accuracy difference? Negligible
```

These aren't edge cases — they're common real-world scenarios. This is where **Vectorless** shines.

So what is **Vectorless**?

Same RAG pipeline. Different retrieval step — **no embeddings, no vector index**.

Vector RAG: Query → **embed()** → **cosine search on vectors** → top-k → LLM

Vectorless: Query → **BM25 / keyword search** → top-k → LLM

✅ What stays the same

- You still chunk your documents
- You still retrieve top-k relevant chunks
- You still inject them as LLM context
- LLM still generates a grounded answer

✅ What changes

- No embedding model needed
- No vector index to maintain
- Use text search you already have
- Works with plain PostgreSQL / SQLite

💡 Vectorless = **RAG with the retrieval step swapped out**. Everything else is identical.

BM25 — The Engine Behind Vectorless

Best Match 25 — a smarter keyword ranking algorithm. Built into Elasticsearch, PostgreSQL, SQLite.

How it scores a document

Query: "cricket batting rules"

Doc A: "batting rules state that..."

→ "batting" appears 3×, "rules" 2×
→ Score: 8.4 ✓ HIGH

Doc B: "the umpire signals a wide"

→ "batting" 0×, "rules" 0×
→ Score: 0.1 ✗ LOW

Doc C: "rules for bowling and batting"

→ both terms present
→ Score: 6.1 ✓ MEDIUM

Ranks by term frequency + how rare the term is across all docs (IDF).

Real example — box cricket bot

```
-- PostgreSQL FTS (BM25-style)
-- No pgvector extension needed!
```

```
SELECT chunk_text
FROM doc_chunks
WHERE to_tsvector('english', chunk_text)
      @@ plainto_tsquery('batting rules')
ORDER BY ts_rank(
  to_tsvector('english', chunk_text),
  plainto_tsquery('batting rules')
) DESC
LIMIT 3;
```

This is **plain PostgreSQL** — no extension, no embedding model, no vector index.

Real Example 1: Support Bot — Error Code Lookup

User asks: "What does error E-4021 mean?" — a case where vector search **fails**.

✗ Vector search gets it wrong

```
embed("error E-4021")  
→ vector is close to:  
  "handling payment errors"  ✗  
  "common API error types"   ✗  
  "error recovery strategies" ✗
```

The model generalises "E-4021"
into a generic "error" concept.
Specific code is lost.

Embeddings are bad at exact IDs, codes, names.

✓ BM25 gets it right

```
BM25("error E-4021")  
→ exact term match:  
  "E-4021: Card declined due  
  to insufficient funds.  
  Ask user to retry with  
  a different card." ✓  
  
Term "E-4021" is rare → high IDF  
→ BM25 ranks it top instantly.
```

Rare, specific terms are BM25's strength.

"E-4021" → **BM25 (PostgreSQL FTS)** → "E-4021: Card declined..." → **LLM** → ✓ "Error E-4021 means your card was declined."

Real Example 2: Internal Wiki — Deploy Question

Engineer asks: "How do we deploy to staging?" — 500 wiki pages, no vector DB.

Step 1 — BM25 retrieves top matches

Keyword search on wiki pages. Fast, no embedding needed.

```
BM25("deploy staging")
→ deploy.md      score: 8.2
→ ci-cd.md       score: 6.1
→ environments.md score: 4.8
```

Step 2 — Feed top chunks to LLM

Top-scored pages become context. LLM answers directly from your docs.

```
Context: [deploy.md content]
User: "How to deploy to staging?"

LLM: "Run ./deploy.sh staging
then verify on Grafana at
grafana.internal/staging" ✓
```

"deploy staging" → **BM25** → top 3 pages → **LLM** →  Grounded answer from your actual wiki

💡 BM25 score is enough to surface the right page — no reranking step needed for structured content like wikis.

Vector vs Vectorless — Pick the Right Tool

Scenario	Use	Why
Error codes, product IDs, SKUs	Vectorless (BM25)	Exact terms — embeddings generalise them away
200 company FAQs	Vectorless (FTS)	Small set, keyword search is enough
Internal wiki (500 pages)	Vectorless (BM25)	Structured content, fast setup, no infra
10,000 research papers	Vector DB	Semantic search across large corpus
Multilingual support docs	Vector DB	Cross-language semantic matching
Image / audio search	Vector DB	No text to keyword-search on
Weekend hackathon / MVP	Vectorless	Ship fast — add vectors later if needed

💡 **Start Vectorless.** Add a vector DB only when you hit a real limit — most projects never do.

Key Takeaways — Vectorless RAG

 **RAG retrieval is a choice** — Dense (vector), Sparse (BM25), or Hybrid. Vector is just one option.

 **Vector search fails** on exact codes/IDs, is expensive to set up, and is overkill for small doc sets.

 **BM25** = keyword ranking built into PostgreSQL, SQLite, Elasticsearch. Zero extra infra.

 **Same RAG pipeline** — only the retrieval step changes. Everything else stays identical.

Vectorless ✓

Questions? 🙋